



NORTHERN UNIVERSITY OF BUSINESS & TECHNOLOGY KHULNA

Department of Computer Science and Engineering



SMART FARMER ASSISTANT

AI-Based Agriculture Support System



Submitted To:

Md. Riaz Mahmud

Assistant Professor, Department of Computer Science and Engineering

Northern University of Business and Technology Khulna

Submitted By:

Team Name: CSE4204-8D-T03

Team Leader: Sakib Foysal Ejarder

ID: 11220320948

Section: 8D

Submission date: 29th June 2026

CSE4204 — Mobile Computing Lab

Week 06 Submission

Database Design

Smart Farmer Assistant

AI-Based Agriculture Support System

Table of Contents

Content	Page
1. Team Information	2
2. Overview	2
3. Database Environment	3
4. Entity-Relationship Diagram	4
5. Table Specifications	5
6. Relationships & Constraints	11
7. Data Validations	12
8. Indexes & Performance	13
9. Migration Strategy	13
10. Conclusion	14

1. Team Information

Field	Details
Team Name	CSE4204-8D-T03
Section	8D
Course	CSE4204 — Software Engineering Lab
Semester	Spring 2026
Institution	Leading University / University (Bangladesh)
GitHub Repo	github.com/sakib-foysal/CSE4204-8D-T03-SmartFarmer-Assistant

1.1 Team Members

#	Name	Student ID	Role
1	Sakib Foysal Ejarder	11220320948	Team Leader / Backend
2	Md. Noyon Sheikh	11220320946	Frontend Developer
3	Sayed Tauhidul Islam	11220320950	AI / ML Engineer
4	Sayed Akib Osman	11220320974	Database / QA

2. Overview

2.1 Project Context

The Smart Farmer Assistant is an AI-based agriculture support system designed to help farmers with:

- Crop disease detection and treatment recommendations
- Chatbot assistance for farming queries
- Market price tracking for crops
- Weather monitoring and flood risk assessment
- Fertilizer recommendations based on soil and crop conditions

2.2 Database Scope

- User management and authentication
- Chatbot interaction history
- Disease detection records with predictions
- Fertilizer recommendation tracking

- Market price data collection
- Weather data logging
- Token revocation for security

3. Database Environment

3.1 Environment Variables

Variable	Purpose
DB_NAME	Database name for the application
DB_USER	Username for database authentication
DB_PASSWORD	Password for database authentication
DB_HOST	Database server hostname/IP
DB_PORT	Database port (default: 5432)

4. Entity-Relationship Diagram

The following diagram shows all entities and their relationships:

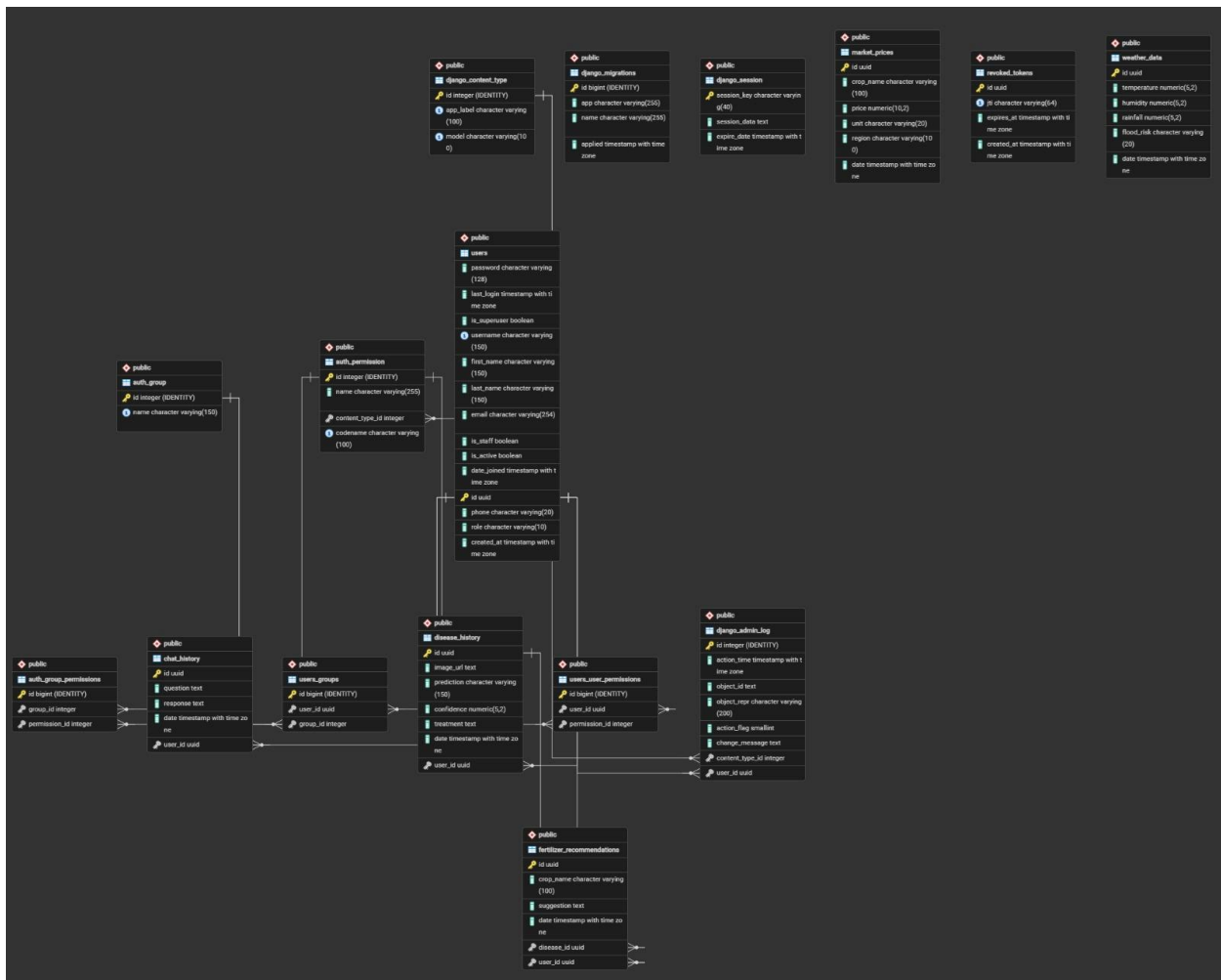


Figure 4.1: Complete ER Diagram showing all tables and relationships in the Smart Farmer Assistant system

5. Table Specifications

5.1 users Table

Stores user account information and authentication details

Column	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier for each user
username	VARCHAR(150)	Unique	Username for login
email	VARCHAR(254)	Unique, Nullable	User email address
password	VARCHAR(255)	Not Null	Hashed password
first_name	VARCHAR(150)	Nullable	User first name
last_name	VARCHAR(150)	Nullable	User last name
phone	VARCHAR(20)	Nullable	Phone number
role	VARCHAR(50)	Default: farmer	User role/type
is_active	BOOLEAN	Default: True	Account status
created_at	TIMESTAMP	Auto	Account creation time

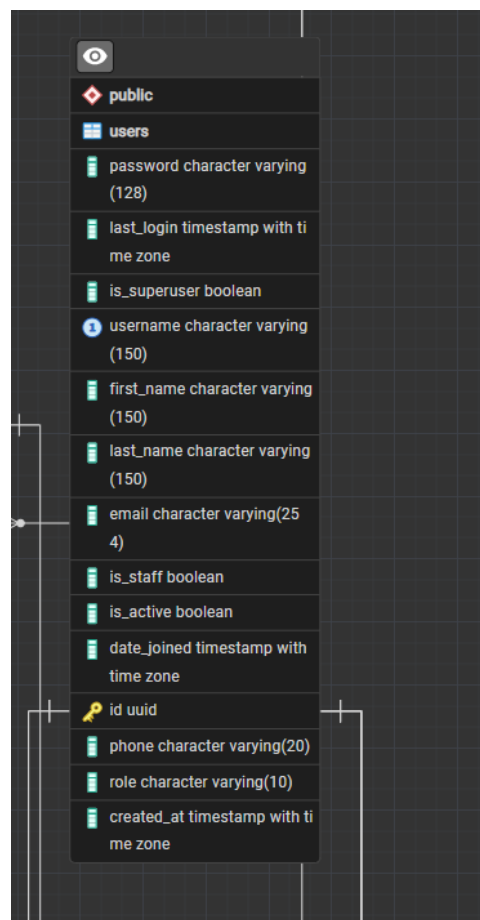


Figure 5.1:users table

5.2 revoked_tokens Table

Maintains list of revoked JWT tokens for logout functionality

Column	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier
jti	VARCHAR(255)	Unique	JWT Token ID
user_id	UUID	Foreign Key	Reference to users table
revoked_at	TIMESTAMP	Auto	Revocation timestamp

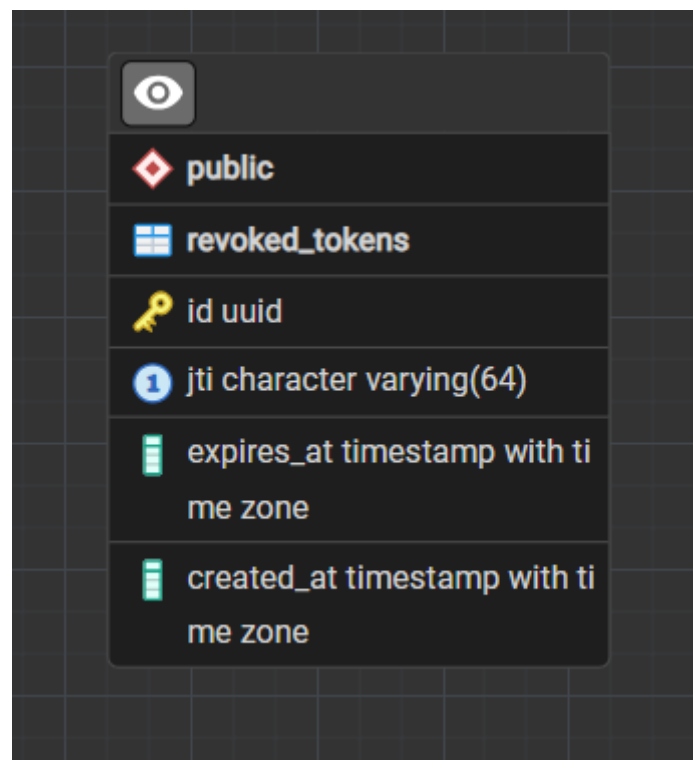


Figure 5.2: revoked_tokens table

5.3 chat_history Table

Stores conversation history between users and chatbot

Column	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier
user_id	UUID	Foreign Key	Reference to users table
question	TEXT	Not Null	User question/query
response	TEXT	Not Null	AI response

created_at	TIMESTAMP	Auto	Query timestamp
------------	-----------	------	-----------------

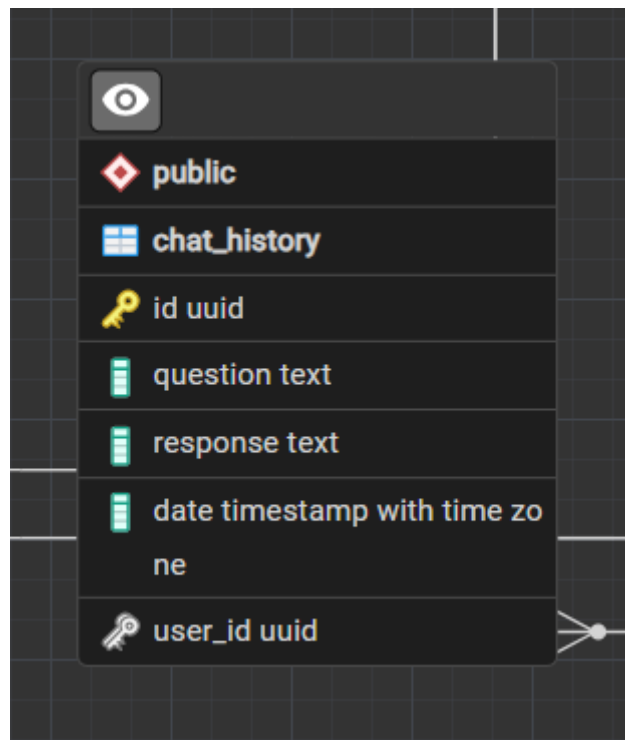


Figure 5.3: chat_history table

5.4 disease_history Table

Tracks crop disease detection results and AI predictions

Column	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier
user_id	UUID	Foreign Key	Reference to users table
image_url	VARCHAR(500)	Nullable	Disease image URL
prediction	VARCHAR(255)	Not Null	Disease name predicted
confidence	DECIMAL(5,2)	0-100	Confidence percentage
treatment	TEXT	Nullable	Recommended treatment
created_at	TIMESTAMP	Auto	Detection timestamp

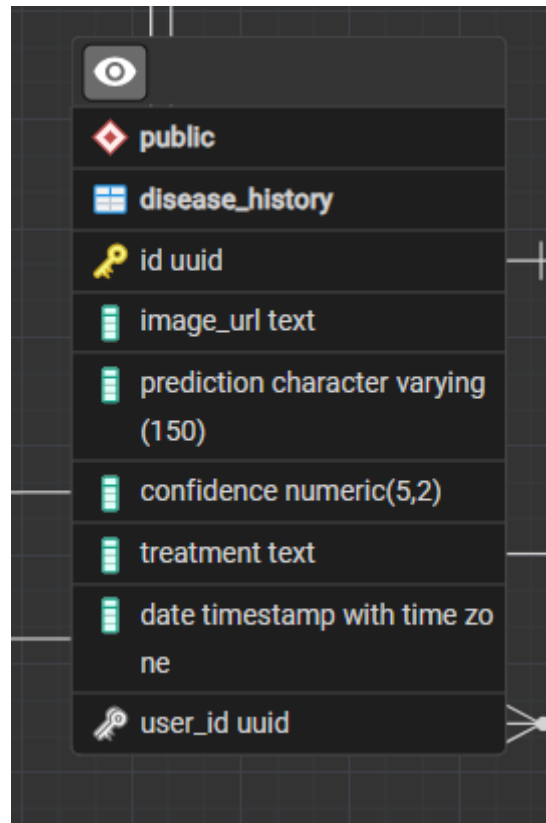


Figure 5.4: disease_history table

5.5 fertilizer_recommendations Table

Stores fertilizer suggestions for users

Column	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier
user_id	UUID	Foreign Key	Reference to users table
disease_history_id	UUID	Foreign Key, Nullable	Reference to disease_history
suggestion	TEXT	Not Null	Fertilizer recommendation
created_at	TIMESTAMP	Auto	Suggestion timestamp

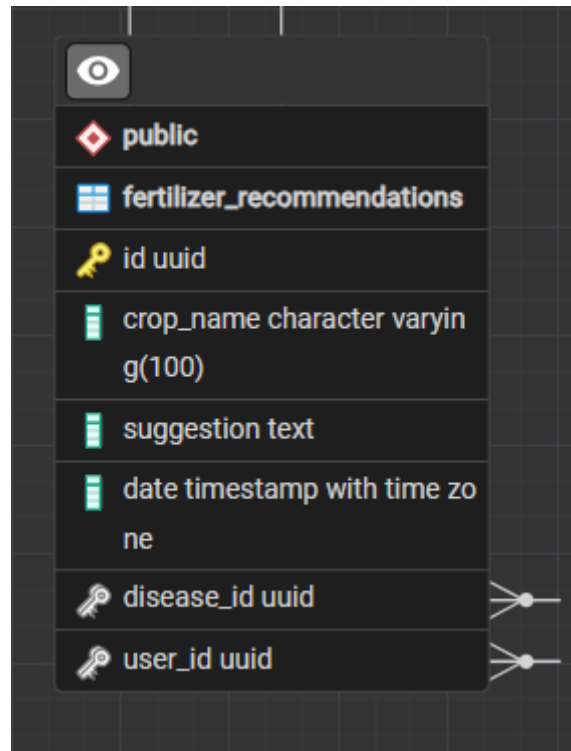


Figure 5.5: fertilizer_recommendations table

5.6 market_prices Table

Maintains crop market price data by region and date

Column	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier
crop_name	VARCHAR(255)	Not Null	Name of crop
price	DECIMAL(10,2)	Not Null	Market price
unit	VARCHAR(50)	Not Null	Price unit (kg, quintal)
region	VARCHAR(255)	Not Null	Geographic region
date	TIMESTAMP	Auto	Price record date

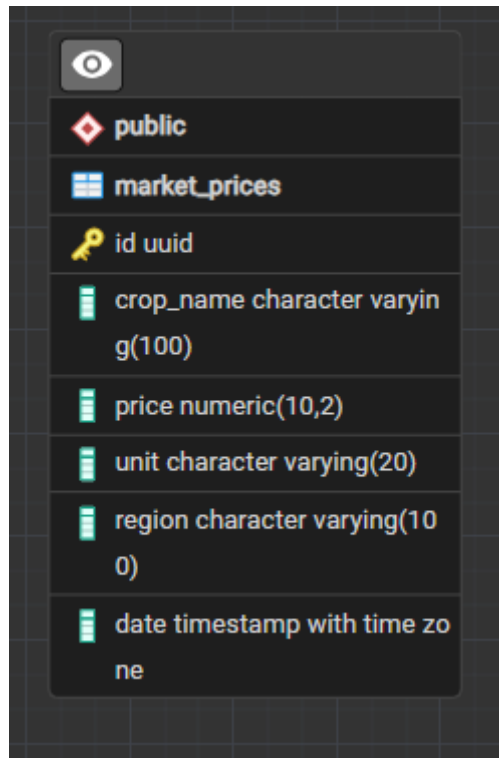


Figure 5.6: market_prices table

5.7 weather_data Table

Stores weather information and flood risk assessment

Column	Data Type	Constraints	Description
id	UUID	Primary Key	Unique identifier
temperature	DECIMAL(5,2)	Range: -50 to 60	Temperature in Celsius
humidity	DECIMAL(5,2)	Range: 0-100	Humidity percentage
rainfall	DECIMAL(8,2)	Non-negative	Rainfall in mm
flood_risk	VARCHAR(20)	low/medium/high	Flood risk level
date	TIMESTAMP	Auto	Data record timestamp

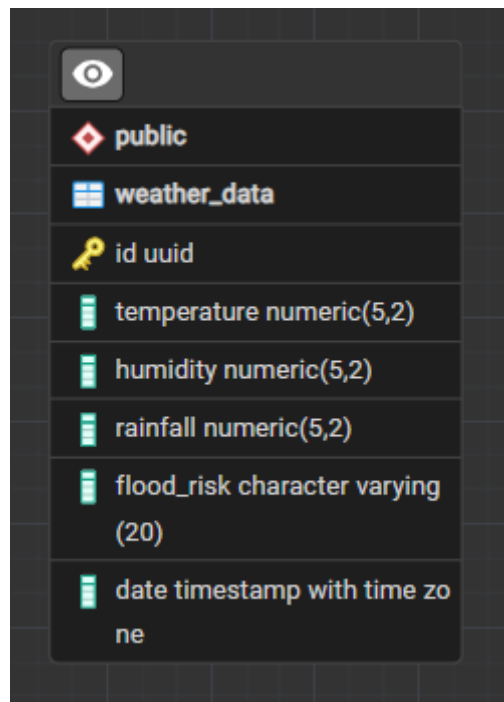


Figure 5.7: weather_data table

6. Relationships & Constraints

Relationship	Type	Description
users ↔ revoked_tokens	One-to-Many	One user can have multiple revoked tokens
users ↔ chat_history	One-to-Many	One user can have multiple chatbot conversations
users ↔ disease_history	One-to-Many	One user can have multiple disease detections
users ↔ fertilizer_recommendations	One-to-Many	One user can receive multiple recommendations
disease_history ↔ fertilizer_recommendations	One-to-Many	One disease detection can lead to multiple fertilizer recommendations

6.1 Foreign Key Constraints

All foreign key relationships have:

- ON DELETE: CASCADE - Deletes related records when parent is deleted
- ON UPDATE: CASCADE - Updates related records when parent is updated

- Referential integrity enforced at database level
- Nullable foreign keys for optional relationships

7. Data Validations

Field	Validation Rule
Username	Unique, required, max 150 characters
Email	Unique when provided, valid email format, max 254 characters
Password	Minimum 8 characters, securely hashed
Phone Number	7-20 digits, numeric only
Disease Confidence	Between 0 and 100 (percentage)
Market Price	Non-negative decimal value
Temperature	Between -50 and 60 degrees Celsius
Humidity	Between 0 and 100 percent
Rainfall	Non-negative decimal value in mm
Flood Risk	Only: low, medium, or high

8. Indexes & Performance

8.1 Primary Key Indexes

UUID primary keys are automatically indexed for fast lookups on all tables

8.2 Foreign Key Indexes

Foreign key columns are automatically indexed for join operations:

- chat_history.user_id
- disease_history.user_id
- fertilizer_recommendations.user_id
- fertilizer_recommendations.disease_history_id
- revoked_tokens.user_id

8.3 Query Optimization

Recommended indexes for common queries:

- users(username, email) - for authentication lookups
- chat_history(user_id, created_at) - for user history with time filtering
- disease_history(user_id, created_at) - for disease records by date
- market_prices(crop_name, region, date) - for price lookups by crop/region

9. Migration Strategy

9.1 Django Migrations

All database changes are managed through Django migrations:

App	Purpose
users app	User model with authentication
revoked_tokens app	Token management for logout
chatbot app	Chat history storage
disease_detection app	Disease prediction records
fertilizer app	Fertilizer recommendations
market app	Market price data
weather app	Weather data storage

9.2 Migration Commands

Create new migrations

```
python manage.py makemigrations
```

Apply migrations to database

```
python manage.py migrate
```

Check migration status

```
python manage.py showmigrations
```

Migrate specific app

```
python manage.py migrate app_name
```

10. Conclusion

The database design for Smart Farmer Assistant follows these principles:

- ✓ Normalized schema to minimize data redundancy
- ✓ UUID primary keys for security and scalability
- ✓ Comprehensive relationships with proper constraints
- ✓ Field validations at both application and database levels
- ✓ Efficient indexing for common query patterns
- ✓ Django ORM migrations for version control
- ✓ Support for future extensions and feature additions

The design supports all core functionality including user management, AI-based recommendations, market data tracking, and weather monitoring. The modular structure allows for easy addition of new features as the system evolves.